1. FUNDAMENTOS DE FRAMEWORKS WEB

1.1. ¿Qué es un Framework?

Un **framework es una estructura base reutilizable** que proporciona una arquitectura predefinida para construir aplicaciones. A diferencia de una simple colección de funciones o bibliotecas, un framework define **cómo se estructura y organiza toda la aplicación**.

Características distintivas:

- **Proporciona arquitectura predefinida**: No empiezas desde cero
- **Incluye librerías y herramientas comunes**: Soluciones a problemas recurrentes
- **Establece buenas prácticas y patrones**: Código estandarizado y mantenible
- **Reduce código repetitivo**: DRY (Don't Repeat Yourself)
- **Facilita el desarrollo rápido**: "No reinventar la rueda"

1.2. Framework vs Librería: Diferencia Fundamental

La diferencia clave es la **Inversión de Control**

Librería:

- **Tú controlas el flujo** del programa
- **Tú llamas** a las funciones de la librería cuando las necesitas
- Es una **herramienta** que usas a tu discreción
- Ejemplo: usar `math.sqrt()` cuando necesites calcular una raíz cuadrada

Framework:

- **El framework controla el flujo** principal
- **El framework te llama** a través de tus implementaciones
- Define el **esqueleto** de la aplicación
- Tú rellenas los espacios específicos (funciones, clases)
- Ejemplo: Flask llama a tu función cuando llega una petición a una ruta específica

Metáforas útiles:

- **Arquitecto vs Albañil**: El framework es el arquitecto (diseña la estructura), tú eres el albañil (rellenas con funcionalidad)
- **Receta vs Ingredientes**: Una librería son ingredientes sueltos; un framework es la receta completa

1.3. ¿Por qué usar Frameworks en la Industria?

Razones empresariales:

- **Tiempo = Dinero**: Aceleran desarrollo, despliegue y mantenimiento

- **Calidad garantizada**: Convenciones probadas, seguridad incluida
- **Escalabilidad**: Diseñados para crecer con el proyecto
- **Mantenibilidad**: Código estandarizado, fácil para nuevos desarrolladores
- **Empleabilidad**: Toda empresa moderna usa frameworks (Django, Flask, React, Angular, Spring)

Ejemplos de uso real:

- Instagram, Spotify, YouTube → **Django**
- Netflix, Reddit, Airbnb → **Flask**
- Facebook, Instagram (frontend) → **React**

2. ARQUITECTURA WEB Y BACKEND

2.1. El Rol del Backend

El **backend** (lado del servidor) es la infraestructura invisible que garantiza el funcionamiento de la aplicación:

Responsabilidades del Backend:

- **Lógica de negocio**: Reglas, cálculos, procesamiento de datos
- **Persistencia de datos**: Almacenamiento y recuperación en bases de datos
- **Seguridad**: Autenticación, autorización, validación de datos
- **Gestión de sesiones**: Seguimiento de usuarios conectados
- **APIs**: Comunicación con otros servicios (internos o externos)

Backend vs Frontend:

Aspecto	Backend	Frontend
Ubicación	Servidor remoto	Navegador del usuario
Visibilidad	Invisible para el usuario	Directamente visible
Tecnologías	Python, Java, PHP, Node.js	HTML, CSS, JavaScript
Responsabilidad	Datos, lógica, seguridad	Presentación, interacción
Ejemplo	Verificar login, calcular precio	Mostrar formulario, animar botón

2.2. Arquitectura de Tres Capas (3-Tier Architecture)

La mayoría de aplicaciones empresariales modernas adoptan esta arquitectura para **separar responsabilidades**, mejorar la **escalabilidad** y aumentar la **seguridad**.

Las tres capas:

1. Capa de Presentación (Presentation Tier)

- **Función**: Interfaz de usuario, recoger entrada y mostrar resultados
- **Tecnologías**: HTML, CSS, JavaScript
- **Ubicación**: Navegador del cliente (frontend)
- **Responsabilidad**: SOLO presentación visual, NO lógica de negocio

2. Capa de Lógica/Aplicación (Logic Tier / Middle Tier)

- **Función**: "Cerebro" de la aplicación, procesamiento y decisiones
- **Tecnologías**: Flask, Django, Spring, Express.js
- **Ubicación**: Servidor de aplicaciones
- **Responsabilidad**:
    - Recibir peticiones de la capa de presentación
    - Ejecutar lógica de negocio
    - Interactuar con la capa de datos
    - Devolver respuestas procesadas

3. Capa de Datos (Data Tier)

- **Función**: Persistencia y gestión de información
- **Tecnologías**: MySQL, PostgreSQL, MongoDB, SQLite
- **Ubicación**: Servidor de base de datos
- **Responsabilidad**:
    - Almacenar datos de forma segura
    - Garantizar integridad referencial
    - Optimizar consultas

¿Por qué separar en capas?

- **Seguridad**: La Capa de Presentación NUNCA accede directamente a la base de datos
- **Escalabilidad**: Cada capa puede escalar independientemente
- **Mantenibilidad**: Cambios en una capa no afectan a las demás
- **Reutilización**: La misma capa de lógica puede servir a múltiples frontends (web, móvil, API)

2.3. El Ciclo Request-Response (HTTP)

Toda aplicación web funciona mediante el **protocolo HTTP** (HyperText Transfer Protocol), que define cómo cliente y servidor se comunican:

Flujo completo:

1. **Cliente genera petición**: Usuario introduce URL o hace clic
2. **Navegador envía HTTP Request**: Viaja por Internet hasta el servidor
3. **Servidor recibe y enruta**: Identifica qué código debe ejecutarse
4. **Framework procesa**: Ejecuta lógica de negocio, consulta BD si es necesario
5. **Servidor genera HTTP Response**: Construye respuesta (HTML, JSON, etc.)

6. **Navegador recibe y renderiza**: Muestra el resultado al usuario

Tipos de peticiones HTTP:

- **GET**: Solicitar datos (leer), parámetros en URL visible
- **POST**: Enviar datos (escribir), datos en cuerpo oculto
- **PUT**: Actualizar recurso completo
- **DELETE**: Eliminar recurso
- **PATCH**: Actualizar parcialmente

---

3. PATRONES DE ARQUITECTURA

3.1. ¿Qué es un Patrón?

Un **patrón** es una **solución reutilizable y probada** a un problema de diseño recurrente. No es código específico, sino una **idea o plantilla conceptual** que se puede aplicar en diferentes contextos y lenguajes.

Lo que NO es un patrón:

- No es código listo para copiar
- No es un algoritmo específico
- No es una librería o framework
- No es un diseño de interfaz

Ejemplos de patrones:

- **MVC**: Separar datos, lógica y presentación
- **Singleton**: Garantizar una única instancia de una clase
- **Factory**: Crear objetos sin especificar su clase exacta
- **Observer**: Notificar automáticamente cambios a dependientes

3.2. Patrón MVC (Modelo-Vista-Controlador)

El **patrón MVC** es uno de los más utilizados en desarrollo web. Separa la aplicación en **tres componentes interconectados**:

Modelo (Model):

- **Representa la estructura de datos** y la lógica de negocio relacionada con datos
- **Gestiona la base de datos**: Crear, leer, actualizar, eliminar (CRUD)
- **Independiente de la presentación**: No sabe nada de HTML o interfaz
- **Ejemplo**: Clase `Movie` con atributos `title`, `director`, `year` y métodos para guardar/recuperar de BD

Vista (View):

- **Presenta información al usuario**: HTML, CSS, JavaScript
- **Recibe datos del Controlador** y los muestra
- **No contiene lógica de negocio**: Solo presentación
- **Ejemplo**: Plantilla HTML que muestra lista de películas en formato visual

Controlador (Controller):

- **Intermediario** entre Modelo y Vista
- **Recibe peticiones del usuario** (rutas HTTP)
- **Invoca al Modelo** para obtener/modificar datos
- **Selecciona la Vista** apropiada y le pasa datos
- **Ejemplo**: Función que recibe petición a `/movies`, obtiene películas del Modelo, y renderiza Vista

Flujo en MVC:

```
None
Usuario → Controlador → Modelo → Base de Datos
                 ↓
Usuario ← Vista ← Controlador (con datos)
```

3.3. Patrón MVT (Model-View-Template) de Django

Django usa una **variante del MVC** llamada **MVT**:

Diferencias con MVC:

- **Model**: Igual que en MVC (capa de datos)
- **View**: En Django contiene la LÓGICA (equivalente al Controller de MVC)
- **Template**: Capa de presentación (equivalente a la View de MVC)
- **Controller**: Implícito en el framework (URL Dispatcher + Middleware)

¿Por qué esta confusión de nombres? Django invierte los nombres "View" y "Controller" respecto al MVC tradicional, pero la separación de responsabilidades es la misma.

3.4. Combinación de Patrones: Arquitectura en Capas + MVC

Las aplicaciones reales **combinan múltiples patrones**:

Ejemplo de arquitectura completa:

- **Capa de Presentación** = Vista/Template (MVC)
- **Capa de Lógica** = Controlador/View (MVC)
- **Capa de Datos** = Modelo (MVC)

Esto permite estructurar sistemas complejos de forma ordenada y escalable.

4. TIPOS DE FRAMEWORKS WEB

4.1. Frameworks Cliente vs Servidor

Frameworks Cliente (Frontend):

- **Ejecución**: Navegador (JavaScript)
- **Ejemplos**: React, Vue.js, Angular, Svelte
- **Responsabilidad**: UI/UX, interactividad, experiencia de usuario
- **Renderizado**: CSR (Client-Side Rendering) - el navegador genera el HTML
- **Datos**: Consume APIs (recibe JSON y lo presenta)

Frameworks Servidor (Backend):

- **Ejecución**: Servidor (Python, PHP, Java, Node.js)
- **Ejemplos**: Flask, Django, Laravel, Spring Boot, Express
- **Responsabilidad**: Lógica de negocio, base de datos, seguridad, APIs
- **Renderizado**: SSR (Server-Side Rendering) - el servidor genera HTML completo
- **Datos**: Procesa, valida y sirve datos

4.2. Frameworks Full-Stack vs Microframeworks

Full-Stack (Ejemplo: Django):

- **Filosofía**: "Baterías incluidas" (batteries included)
- **Características**:
  - Incluye TODO lo necesario: ORM, sistema de plantillas, admin panel, autenticación, formularios, seguridad
  - Arquitectura rígida y estandarizada (MVT)
  - Convención sobre configuración
- **Ventajas**:
  - Desarrollo muy rápido para proyectos estándar
  - Menos decisiones que tomar
  - Equipo grande puede colaborar fácilmente (estructura uniforme)
- **Desventajas**:
  - Curva de aprendizaje más pronunciada
  - Menos flexibilidad
  - "Overhead" para proyectos simples

Microframeworks (Ejemplo: Flask):

- **Filosofía**: Minimalista, "microkernel"
- **Características**:
  - Solo incluye lo esencial: routing, manejo de peticiones, templates básicos
  - Sin arquitectura impuesta
  - Configuración sobre convención

    - Funcionalidades avanzadas mediante extensiones
- **Ventajas**:
    - Curva de aprendizaje suave
    - Máxima flexibilidad arquitectónica
    - Perfecto para aprender conceptos web fundamentales
    - Ligero y rápido
- **Desventajas**:
    - Proyectos grandes requieren más decisiones de diseño
    - Necesitas buscar e integrar extensiones manualmente
    - Más responsabilidad en el desarrollador

---

5. FLASK EN PROFUNDIDAD

5.1. Filosofía y Posicionamiento de Flask

Flask se posiciona como el **"punto medio perfecto"** en el ecosistema Python:

Entre dos extremos:

- **WSGI puro**: Demasiado bajo nivel, reinventas todo
- **Django**: Demasiado opinionado, impone decisiones

Flask es el "gateway drug":

- La mayoría de desarrolladores Python web empiezan aquí
- Estándar de facto para enseñar desarrollo web en Python
- Base conceptual para aprender frameworks más complejos

5.2. Componentes Core de Flask

Flask se construye sobre **dos bibliotecas fundamentales**:

1. Werkzeug

- **Tipo**: Biblioteca WSGI completa
- **Función**: Gestiona el ciclo Request/Response
- **Responsabilidades**:
    - Parsear peticiones HTTP entrantes
    - Construir respuestas HTTP salientes
    - Manejo de cookies, sesiones
    - Utilidades de desarrollo (debugger, reloader)

2. Jinja2

- **Tipo**: Motor de plantillas
- **Función**: Renderizar HTML dinámico

- **Responsabilidades**:
    - Inyectar datos en HTML
    - Control de flujo en templates (if, for)
    - Herencia de plantillas (DRY)
    - Filtros y funciones personalizadas

5.3. El Estándar WSGI (Web Server Gateway Interface)

Problema histórico: Antes de WSGI, cada framework Python solo funcionaba con servidores específicos → ecosistema fragmentado.

Solución WSGI (PEP 3333):

- **Estándar de interfaz** entre servidores web y aplicaciones Python
- **Garantiza portabilidad**: Cualquier servidor WSGI puede ejecutar cualquier aplicación WSGI
- **Capa de abstracción**: El código de la app es agnóstico al servidor final

Arquitectura WSGI en producción:

```
None
Internet → Nginx (Web Server) → Gunicorn (WSGI Gateway) →
Flask (Aplicación)
```

¿Por qué esta arquitectura?

- **Nginx**: Maneja archivos estáticos, SSL, balanceo de carga
- **Gunicorn**: Gestiona múltiples workers Python, rendimiento
- **Flask**: Lógica de la aplicación

Importante: El servidor de desarrollo de Flask (`app.run()`) es SOLO para pruebas, NUNCA para producción.

5.4. Gestión de Dependencias: Entornos Virtuales

Problema: Diferentes proyectos requieren diferentes versiones de las mismas bibliotecas → conflictos inevitables si se usa Python global.

Solución: Entornos Virtuales (venv)

- **Espacio aislado** para cada proyecto
- **Instalaciones locales**: Las bibliotecas viven dentro del proyecto
- **Sin contaminación**: No afecta a otros proyectos ni al Python global
- **Reproducibilidad**: `requirements.txt` permite replicar el entorno exacto

Herramientas disponibles:

- **venv**: Built-in en Python 3.3+, simple, estándar
- **virtualenv**: Más potente, compatible con Python 2
- **Pipenv**: Gestión moderna, combina pip + venv, usa Pipfile
- **Poetry**: Gestión completa de dependencias y publicación de paquetes
- **Conda**: Gestiona también dependencias nativas (muy usado en ciencia de datos)

5.5. Extensiones de Flask

Flask sigue el principio de **"microkernel + extensiones"**. Las extensiones se integran de forma transparente:

Extensiones Core (oficiales):

- **Flask-SQLAlchemy**: ORM para bases de datos relacionales
- **Flask-Login**: Sistema de autenticación de usuarios
- **Flask-WTF**: Generación y validación de formularios, protección CSRF
- **Flask-Mail**: Envío de emails

Extensiones Populares (third-party):

- **Flask-RESTful**: Construcción rápida de APIs REST
- **Flask-JWT**: JSON Web Tokens para autenticación API
- **Flask-Migrate**: Migraciones de base de datos (como Alembic)
- **Flask-CORS**: Cross-Origin Resource Sharing para APIs

Ventaja del sistema de extensiones: Instalas SOLO lo que necesitas → aplicaciones ligeras y personalizadas.

---

6. ROUTING Y MANEJO DE PETICIONES

6.1. El Decorador @app.route()

El **decorador** es el mecanismo fundamental de Flask para definir cómo se manejan las URLs:

Función del decorador:

- **Enlaza una URL** con una función Python
- **Define métodos HTTP** permitidos (GET, POST, etc.)
- **Captura parámetros** dinámicos de la URL

Conceptualmente: El decorador "registra" la función en una tabla interna de rutas. Cuando llega una petición, Flask busca en esta tabla qué función debe ejecutar.

6.2. Rutas Estáticas vs Dinámicas

Rutas Estáticas:

- URL fija, siempre igual
- Ejemplo: `/about`, `/contact`
- Uso: Páginas con contenido predecible

Rutas Dinámicas:

- Incluyen **placeholders** que capturan valores variables
- Sintaxis: `<tipo:nombre>`
- Ejemplo: `/movie/<int:mid>` captura un entero
- Uso: Páginas con contenido basado en parámetros (detalles, perfiles, búsquedas)

Tipos de conversores disponibles:

- `<int:id>`: Números enteros
- `<float:precio>`: Números decimales
- `<path:ruta>`: Acepta barras (para subcarpetas)
- `<string:nombre>`: Texto (por defecto)
- `<uuid:id>`: Identificadores UUID

6.3. El Objeto request

`request` es un **objeto de contexto global** que contiene toda la información de la petición HTTP actual:

Atributos principales:

- **request.method**: Tipo de petición (`'GET'`, `'POST'`, etc.)
- **request.form**: Datos de formulario (POST)
- **request.args**: Parámetros de URL (GET) - ej: `?search=matrix`
- **request.files**: Archivos subidos
- **request.cookies**: Cookies del navegador
- **request.headers**: Cabeceras HTTP
- **request.json**: Datos JSON (para APIs)

Contexto de aplicación: `request` es un proxy inteligente que automáticamente se refiere a la petición actual del thread/worker que la procesa → permite que Flask maneje múltiples peticiones simultáneas de forma segura.

---

7. MOTORES DE PLANTILLAS: JINJA2

7.1. ¿Por qué Motores de Plantillas?

Problema: Generar HTML dinámico mezclando código Python y strings HTML es caótico, ilegible y propenso a errores.

Solución: Los **motores de plantillas** permiten **separar presentación de lógica**:

- **Diseñadores** trabajan en HTML/CSS
- **Desarrolladores** pasan datos desde Python
- **Mantenibilidad** mejorada: cada uno en su dominio

7.2. Sintaxis de Jinja2: Los Tres Delimitadores

1. Expresiones {{ }}

- **Imprime valores** en el HTML
- Evalúa expresiones Python
- Ejemplo: `{{ usuario.nombre }}`, `{{ 2 + 2 }}`

2. Sentencias {% %}

- **Estructuras de control**: if, for, block
- **No genera salida** directamente
- Ejemplo: `{% if edad >= 18 %}`, `{% for item in lista %}`

3. Comentarios {# #}

- **Notas internas** que no aparecen en el HTML final
- Ejemplo: `{# TODO: Mejorar diseño #}`

7.3. Herencia de Plantillas

Principio DRY (Don't Repeat Yourself): Evitar duplicar el esqueleto común (header, footer, menú) en cada página.

Mecanismo:

Plantilla Base (base.html):

- Define la **estructura general** del sitio
- Incluye elementos comunes (navegación, scripts)
- Declara **bloques** (placeholders) que las plantillas hijas pueden rellenar

Plantillas Hijas:

- **Extienden** la plantilla base con `{% extends "base.html" %}`
- **Sobrescriben bloques** específicos con su contenido único
- Heredan automáticamente todo lo demás

Ventajas:

- **Un cambio en base.html** afecta a todas las páginas (actualización centralizada)

- **Consistencia visual** garantizada
- **Desarrollo más rápido**: Solo defines lo específico

7.4. Contexto: Pasar Datos del Backend

El **contexto** es un **diccionario Python** que transporta datos desde la lógica (Flask) hasta la presentación (Jinja2):

Proceso:

1. La función de vista prepara datos (consulta BD, cálculos)
2. Crea un diccionario con los datos necesarios
3. Lo pasa a `render_template()` como argumentos
4. Jinja2 puede acceder a estas variables con `{{ variable }}`

Ejemplo conceptual:

```
None
Backend: context = {'peliculas': [...], 'usuario': 'Ana'}
Template: acceso mediante {{ peliculas }} y {{ usuario }}
```

---

8. FORMULARIOS Y VALIDACIÓN

8.1. Métodos HTTP: GET vs POST

GET (Solicitar datos - Lectura):

- **Parámetros en la URL** (visibles)
- **Idempotente**: Múltiples peticiones idénticas = mismo resultado
- **Cacheable**: Navegadores pueden cachear
- **Uso**: Navegar páginas, búsquedas, filtros
- **Ejemplo**: `/movies?genre=action&year=2020`

POST (Enviar datos - Escritura):

- **Datos en el cuerpo** de la petición (ocultos)
- **No idempotente**: Puede causar cambios (crear registro en BD)
- **No cacheable**
- **Uso**: Enviar formularios, crear/actualizar recursos
- **Ejemplo**: Enviar mensaje de contacto, registro de usuario

Regla de oro:

- GET para **leer/consultar**
- POST para **escribir/modificar**

8.2. Patrón PRG (Post-Redirect-Get)

Problema: Si después de un POST muestras directamente una página, al recargar (F5) el navegador reenvía el formulario → duplicación de acciones (doble registro, doble pago, etc.)

Solución: Patrón PRG

1. Usuario envía formulario (**POST**)
2. Servidor procesa datos
3. Servidor **redirige** (código HTTP 302) a otra URL
4. Navegador hace nueva petición (**GET**) a esa URL
5. Servidor responde con página de éxito

Ventaja: Recargar la página solo repite el GET final, no el POST original → seguridad.

8.3. Flash Messages

Concepto: Mensajes temporales que se muestran al usuario **una sola vez** tras una acción (éxito, error, advertencia).

Funcionamiento:

- Se **almacenan en la sesión** del usuario
- Se **recuperan y eliminan** en el siguiente renderizado
- Requieren **SECRET_KEY** para cifrar la sesión

Categorías comunes:

- `success`: Acción completada correctamente
- `error`: Ha ocurrido un error
- `warning`: Advertencia que no bloquea
- `info`: Información neutral

8.4. SECRET_KEY

Función: Clave criptográfica que Flask usa para **firmar datos sensibles** (sesiones, flash messages, cookies firmadas).

¿Por qué es necesaria?

- **Seguridad**: Evita que usuarios manipulen sesiones
- **Integridad**: Garantiza que los datos no han sido alterados
- **Autenticidad**: Verifica que los datos provienen del servidor

Buenas prácticas:

- NUNCA hardcodear en el código fuente
- Usar variables de entorno en producción
- Generar claves aleatorias largas y complejas

---

9. REDIRECCIONES Y GENERACIÓN DE URLs

9.1. redirect()

Función: Devuelve una respuesta HTTP con código de redirección (302 por defecto) que indica al navegador que visite otra URL.

Uso típico:

- Después de procesar un formulario (patrón PRG)
- Redirigir páginas antiguas a nuevas
- Forzar autenticación (enviar a login)

9.2. url_for()

Problema: Hardcodear URLs (`return redirect('/movies')`) es frágil: si cambias la ruta, rompes todas las referencias.

Solución: url_for() Genera URLs **basándose en el nombre de la función**, no en la ruta:

Ventajas:

- **Flexibilidad**: Cambias la ruta en un solo lugar (`@app.route`)
- **Mantenibilidad**: El código se adapta automáticamente
- **Parámetros dinámicos**: Inyecta valores en rutas dinámicas
- **URLs correctas**: Respeta el prefijo de la aplicación (si existe)

---

10. MANEJO DE ERRORES

10.1. abort()

Función: Interrumpe el procesamiento inmediatamente y devuelve un **código de error HTTP** específico.

Códigos comunes:

- **404 Not Found**: Recurso no existe
- **403 Forbidden**: No tiene permisos
- **500 Internal Server Error**: Error del servidor
- **401 Unauthorized**: No autenticado

Uso: Validar condiciones críticas (recurso existe, usuario tiene permisos, etc.) y abortar si fallan.

10.2. Páginas de Error Personalizadas

Problema: Las páginas de error por defecto de Flask son feas y no mantienen la identidad visual del sitio.

Solución: errorhandler() Decorador que intercepta errores específicos y renderiza plantillas personalizadas.

Ventajas:

- **Consistencia visual**: Mantienen el diseño del sitio
- **Mejor UX**: Mensajes amigables, enlaces útiles
- **Branding**: Página de error como oportunidad de marca

---

11. COMPARACIÓN: FLASK VS DJANGO

11.1. Criterios de Selección

¿Cuándo elegir Flask?

- Proyectos pequeños/medianos
- APIs REST (especialmente con Flask-RESTful)
- Microservicios independientes
- Prototipos rápidos (MVPs)
- Máximo control arquitectónico
- Recursos limitados (IoT, Edge Computing)
- Aprendizaje de conceptos web fundamentales

¿Cuándo elegir Django?

- Proyectos grandes con muchas funcionalidades
- Necesitas panel de administración robusto
- Equipos grandes o juniors (estructura guía)
- Aplicaciones con muchos modelos de datos relacionales
- Plazos ajustados (baterías incluidas aceleran desarrollo)
- CMS, e-commerce, plataformas sociales

11.2. Tabla Comparativa Detallada

Aspecto	Flask	Django
Tipo	Microframework	Full-Stack
Filosofía	Configuración sobre convención	Convención sobre configuración
Arquitectura	Libre (no impuesta)	MVT rígido

ORM	No incluido (SQLAlchemy externo)	Django ORM integrado y potente
Admin Panel	No incluido	Automático y personalizable
Autenticación	Extensión (Flask-Login)	Sistema completo integrado
Formularios	Extensión (Flask-WTF)	Sistema integrado con validación
Seguridad	Responsabilidad del desarrollador	Protecciones por defecto
Curva Aprendizaje	Suave inicialmente	Pronunciada pero luego productivo
Flexibilidad	Máxima	Limitada por convenciones
Velocidad Desarrollo	Lenta en proyectos grandes	Muy rápida en proyectos estándar
Escalabilidad	Buena (con diseño correcto)	Excelente (probado en Instagram)
Comunidad	Grande	Muy grande

11.3. Persistencia de Datos

Flask:

- Sin ORM incluido → **máxima libertad**
- Ideal para **NoSQL** (MongoDB, Redis)
- Ideal para **APIs sin BD** (gateways, proxies)
- Si necesitas SQL: integrar **SQLAlchemy** (muy potente pero más complejo)

Django:

- **Django ORM** integrado y maduro
- Optimizado para **BD relacionales** (PostgreSQL, MySQL, SQLite)
- **Migraciones automáticas**: Cambios en modelos → scripts SQL
- **Query API** intuitiva en Python puro

11.4. Seguridad

Django:

- **Protección CSRF** activada por defecto
- **SQL Injection** prevenida por el ORM
- **XSS** mitigado por auto-escape en templates
- **Clickjacking** protección por cabeceras

- **Gestión segura de contraseñas** (hashing automático)

Flask:

- **Responsabilidad del desarrollador**
- Debe implementar/integrar protecciones manualmente
- Flask-WTF ayuda con CSRF
- Más flexible pero más peligroso si no tienes experiencia

---

12. CONCEPTOS AVANZADOS

12.1. Blueprints de Flask

Concepto: Mecanismo de **modularización** para proyectos Flask grandes.

Función:

- Agrupa rutas, plantillas y recursos relacionados
- Permite **organizar la aplicación por funcionalidad** (auth

Reintentar
L

Continuar

12. CONCEPTOS AVANZADOS (Continuación)

12.1. Blueprints de Flask (Continuación)

Función:

- Agrupa rutas, plantillas y recursos relacionados
- Permite **organizar la aplicación por funcionalidad** (auth, blog, api, admin)
- **Reutilizable**: Un blueprint puede usarse en múltiples aplicaciones
- **Namespace**: Evita conflictos de nombres entre módulos

Analogía con Django: Los **Blueprints de Flask** son conceptualmente similares a las **Apps de Django**, pero:

- **Más simples**: Menos estructura obligatoria
- **Más ligeros**: Menos convenciones
- **Mismo objetivo**: Modularidad y reutilización

¿Cuándo usar Blueprints?

- Proyecto supera 5-10 rutas

- Múltiples funcionalidades distintas (usuarios, productos, blog)
- Equipo trabajando en paralelo (cada uno su módulo)
- Necesitas mantener código organizado y escalable

Estructura típica con Blueprints:

```
None
app/
├── __init__.py          # Crea app, registra blueprints
├── auth/                # Blueprint de autenticación
│   ├── __init__.py
│   └── routes.py
├── blog/                # Blueprint de blog
│   ├── __init__.py
│   └── routes.py
└── api/                 # Blueprint de API
    ├── __init__.py
    └── routes.py
```

12.2. Configuración de la Aplicación Flask

app.config: Diccionario especial para almacenar **configuraciones globales** de la aplicación.

Configuraciones típicas:

- **SECRET_KEY**: Clave criptográfica para sesiones
- **DEBUG**: Modo desarrollo (NUNCA en producción)
- **TESTING**: Activa modo pruebas
- **DATABASE_URI**: Conexión a base de datos
- **UPLOAD_FOLDER**: Directorio para archivos subidos
- **MAX_CONTENT_LENGTH**: Tamaño máximo de uploads

Buenas prácticas de configuración:

1. Separar configuraciones por entorno:

- **Development**: Debug activado, BD local, logs verbosos
- **Testing**: BD en memoria, datos de prueba
- **Production**: Debug desactivado, BD remota, seguridad máxima

2. Usar variables de entorno: Nunca hardcodear credenciales → leerlas de variables de entorno:

- Desarrollo: archivo `.env` (nunca en git)
- Producción: Variables del servidor/contenedor

3. Objetos de configuración: Crear clases para cada entorno (DevelopmentConfig, ProductionConfig) → más mantenible.

12.3. Contextos en Flask

Flask maneja dos tipos de **contextos** (información disponible durante el procesamiento):

1. Contexto de Aplicación (Application Context):

- Información sobre la **aplicación Flask en sí**
- Variables: `current_app`, `g`
- **current_app**: Referencia a la instancia de la aplicación
- **g**: Almacenamiento temporal durante una petición (útil para compartir datos entre funciones)

2. Contexto de Petición (Request Context):

- Información sobre la **petición HTTP actual**
- Variables: `request`, `session`
- **request**: Datos de la petición (form, args, method, etc.)
- **session**: Datos persistentes del usuario entre peticiones

¿Por qué contextos? Flask puede manejar **múltiples peticiones simultáneas**. Los contextos aseguran que cada petición acceda a SUS datos sin interferir con otras → thread-safety.

12.4. Sesiones (Sessions)

Concepto: Las **sesiones** permiten **almacenar información del usuario entre peticiones** (el protocolo HTTP es stateless por naturaleza).

¿Cómo funcionan?

1. Servidor crea un **session ID** único para cada usuario
2. Lo envía al navegador como **cookie**
3. Navegador incluye esta cookie en cada petición subsiguiente
4. Servidor identifica al usuario y recupera sus datos de sesión

Almacenamiento en Flask:

- Por defecto: **Client-side** (datos cifrados en cookie)
- Ventaja: Sin carga en servidor
- Desventaja: Tamaño limitado (4KB), datos viajan con cada petición

Alternativas (con extensiones):

- **Flask-Session**: Sesiones server-side (Redis, Memcached, BD)
- Ventaja: Sin límite de tamaño, más seguro
- Desventaja: Requiere infraestructura adicional

Uso típico de sesiones:

- Mantener usuario logueado
- Carrito de compras
- Preferencias del usuario (idioma, tema)
- Datos de formularios multipaso

12.5. Middlewares

Concepto: Los **middlewares** son componentes que **interceptan y procesan peticiones/respuestas** antes de que lleguen a las rutas o después de que las rutas generen respuesta.

Funciones típicas de middlewares:

- **Autenticación**: Verificar si el usuario está logueado
- **Logging**: Registrar cada petición
- **Compresión**: Comprimir respuestas (gzip)
- **CORS**: Gestionar peticiones cross-origin
- **Rate Limiting**: Limitar peticiones por IP
- **Manejo de errores global**: Capturar excepciones

En Flask: Se implementan mediante decoradores especiales:

- `@app.before_request`: Ejecuta ANTES de cada petición
- `@app.after_request`: Ejecuta DESPUÉS de cada petición
- `@app.teardown_request`: Limpieza (aunque haya error)

Ejemplo conceptual:

```
None
Petición → before_request → Ruta → after_request → Respuesta
```

---

13. APIS Y ARQUITECTURAS MODERNAS

13.1. ¿Qué es una API?

API (Application Programming Interface): Interfaz que permite que **dos aplicaciones se comuniquen** entre sí mediante un conjunto de reglas y protocolos definidos.

En contexto web: Una **API web** expone funcionalidades del backend mediante URLs (endpoints) que devuelven **datos estructurados** (generalmente JSON), en lugar de HTML.

Diferencia clave:

- **Web tradicional**: Servidor devuelve **HTML renderizado** (para humanos)
- **API web**: Servidor devuelve **datos puros JSON** (para otras aplicaciones)

13.2. REST (Representational State Transfer)

REST es un **estilo arquitectónico** para diseñar APIs web, basado en principios específicos:

Principios REST:

1. Recursos identificados por URLs: Cada "cosa" (recurso) tiene una URL única:

- `/movies` → Colección de películas
- `/movies/5` → Película con ID 5
- `/users/123/posts` → Posts del usuario 123

2. Métodos HTTP para operaciones CRUD:

- **GET**: Leer/consultar (Read)
- **POST**: Crear (Create)
- **PUT**: Actualizar completo (Update)
- **PATCH**: Actualizar parcial (Update)
- **DELETE**: Eliminar (Delete)

3. Representaciones (JSON, XML): Los recursos se representan en formato intercambiable, típicamente JSON.

4. Stateless (Sin estado): Cada petición contiene TODA la información necesaria → el servidor no almacena contexto entre peticiones.

5. Interfaz uniforme: URLs, métodos y formatos consistentes → predecible y fácil de usar.

Ejemplo de API REST:

```None
GET    /api/movies       → Listar todas las películas
GET    /api/movies/5     → Obtener película 5
POST   /api/movies       → Crear nueva película
PUT    /api/movies/5     → Actualizar película 5 (completa)
PATCH  /api/movies/5     → Actualizar película 5 (parcial)
DELETE /api/movies/5     → Eliminar película 5
```

13.3. GraphQL

GraphQL es una **alternativa a REST** desarrollada por Facebook.

Diferencia fundamental:

- **REST**: Múltiples endpoints, cada uno devuelve estructura fija
- **GraphQL**: Un solo endpoint, el cliente especifica exactamente qué datos necesita

Ventajas de GraphQL:

- **Sin overfetching**: No recibes datos innecesarios
- **Sin underfetching**: Una query obtiene todo lo necesario (sin múltiples peticiones)
- **Tipado fuerte**: Schema define tipos y validaciones
- **Introspección**: El API se autodocumenta

Desventajas:

- Más complejo de implementar
- Caching más difícil
- Curva de aprendizaje mayor

13.4. SOAP (Simple Object Access Protocol)

SOAP es un **protocolo más antiguo y rígido** para APIs:

Características:

- Usa **XML** exclusivamente (verboso)
- Requiere **WSDL** (Web Services Description Language) → contrato estricto
- Más complejo pero más robusto para sistemas empresariales
- Típicamente usa solo POST (no aprovecha HTTP semántico)

Cuándo SOAP: Sistemas bancarios, legacy empresarial, cuando se necesita transaccionalidad estricta.

Tendencia actual: REST y GraphQL han reemplazado a SOAP en la mayoría de casos modernos.

13.5. Arquitectura de Microservicios

Concepto: En lugar de una aplicación monolítica grande, dividir la funcionalidad en **servicios pequeños e independientes** que se comunican entre sí.

Características de Microservicios:

- **Independencia**: Cada servicio tiene su propia BD y código
- **Despliegue independiente**: Actualizar un servicio no afecta a otros
- **Tecnología heterogénea**: Cada servicio puede usar lenguajes/frameworks diferentes
- **Escalabilidad granular**: Escalar solo los servicios que lo necesitan

Ejemplo: E-commerce dividido en:

- Servicio de Usuarios (Flask)
- Servicio de Productos (Node.js)
- Servicio de Pagos (Java)
- Servicio de Notificaciones (Python)

Comunicación entre microservicios:

- APIs REST/GraphQL
- Message Queues (RabbitMQ, Kafka)
- gRPC (comunicación binaria eficiente)

Ventajas:

- Equipos autónomos
- Fallos aislados (un servicio caído no tumba todo)
- Escalabilidad eficiente

Desventajas:

- Complejidad operacional (muchos servicios que gestionar)
- Debugging más difícil
- Latencia de red entre servicios

---

14. RENDERIZADO: SSR vs CSR

14.1. SSR (Server-Side Rendering)

Concepto: El **servidor genera el HTML completo** y lo envía al navegador listo para mostrar.

Proceso:

1. Navegador solicita página
2. Servidor ejecuta lógica (consulta BD)
3. Servidor renderiza plantilla con datos
4. Servidor envía HTML completo
5. Navegador muestra directamente

Ventajas:

- **SEO excelente**: Motores de búsqueda ven contenido completo
- **Primera carga rápida**: El HTML ya está generado
- **Funciona sin JavaScript**: Accesibilidad garantizada
- **Menor carga en el cliente**: El servidor hace el trabajo pesado

Desventajas:

- **Interactividad limitada**: Cada acción requiere nueva petición al servidor
- **Mayor carga del servidor**: Renderiza para cada usuario
- **Experiencia menos fluida**: Recargas completas de página

Frameworks SSR:

- Flask/Django con Jinja2
- Next.js (React con SSR)
- Nuxt.js (Vue con SSR)

14.2. CSR (Client-Side Rendering)

Concepto: El **navegador recibe una página HTML mínima** y JavaScript que genera el contenido dinámicamente.

Proceso:

1. Navegador solicita página
2. Servidor envía HTML básico + JavaScript
3. Navegador ejecuta JavaScript
4. JavaScript solicita datos al API (JSON)
5. JavaScript genera y renderiza HTML

Ventajas:

- **Experiencia fluida**: Sin recargas, transiciones suaves
- **Interactividad rica**: Aplicaciones complejas tipo desktop
- **Menor carga del servidor**: Solo sirve datos JSON
- **Separación frontend/backend**: Equipos independientes

Desventajas:

- **SEO complicado**: Motores de búsqueda pueden no ver contenido
- **Primera carga lenta**: Descargar y ejecutar JavaScript tarda
- **Requiere JavaScript**: No funciona si está deshabilitado
- **Mayor carga del cliente**: Dispositivos lentos sufren

Frameworks CSR:

- React (puro)
- Vue.js
- Angular
- Svelte

14.3. Híbridos y Tendencias Modernas

SSG (Static Site Generation):

- Genera HTML en **tiempo de build**, no en cada petición
- Mejor rendimiento (HTML estático)

- Frameworks: Next.js, Gatsby, Eleventy

ISR (Incremental Static Regeneration):

- Híbrido SSG + SSR
- Genera estáticamente pero puede regenerar bajo demanda
- Next.js popularizó este enfoque

Hydration:

- Envía HTML renderizado (SSR) + JavaScript para interactividad
- "Lo mejor de ambos mundos"
- Primera carga rápida + interactividad después

---

15. SEGURIDAD WEB FUNDAMENTAL

15.1. Inyección SQL

Ataque: Insertar código SQL malicioso en inputs para manipular la base de datos.

Ejemplo vulnerable:

```
None
query = f"SELECT * FROM users WHERE username = '{user_input}'"
```

Si `user_input = "admin' OR '1'='1"` → devuelve todos los usuarios.

Prevención:

- **Usar ORM**: Django ORM, SQLAlchemy (escapan automáticamente)
- **Prepared statements**: Parámetros separados de la query
- **NUNCA concatenar input del usuario en SQL**

15.2. XSS (Cross-Site Scripting)

Ataque: Inyectar JavaScript malicioso que se ejecuta en el navegador de otros usuarios.

Ejemplo vulnerable: Mostrar comentario de usuario sin escapar:
`<script>alert('Hacked!')</script>`

Prevención:

- **Auto-escape en templates**: Jinja2 escapa por defecto
- **Validar y sanitizar inputs**
- **Content Security Policy (CSP)**: Header HTTP que restringe scripts

15.3. CSRF (Cross-Site Request Forgery)

Ataque: Engañar al navegador del usuario para que ejecute acciones no deseadas en un sitio donde está autenticado.

Ejemplo: Usuario logueado en banco. Visita sitio malicioso que contiene:

html

```
HTML
<img src="https://banco.com/transfer?to=atacante&amount=1000">
```

El navegador envía la petición CON las cookies de autenticación → transferencia no autorizada.

Prevención:
- **CSRF Tokens**: Valores aleatorios únicos por sesión en formularios
- **SameSite cookies**: Restringen envío de cookies en peticiones cross-site
- **Verificar Referer/Origin headers**

Django y Flask-WTF incluyen protección CSRF por defecto.

15.4. Gestión Segura de Contraseñas

NUNCA almacenar contraseñas en texto plano.

Proceso correcto:
1. **Hash**: Función unidireccional (no se puede revertir)
2. **Salt**: Valor aleatorio único por usuario (evita rainbow tables)
3. **Key Stretching**: Algoritmo lento (dificulta fuerza bruta)

Algoritmos recomendados:
- **bcrypt**: Estándar de facto
- **Argon2**: Ganador de competición Password Hashing Competition
- **PBKDF2**: Usado por Django

```
**NUNCA usar:**
- MD5, SHA1, SHA256 directamente (demasiado rápidos)

### 15.5. HTTPS y Cifrado

**HTTP vs HTTPS:**
- **HTTP**: Datos viajan en texto plano → interceptables
- **HTTPS**: Datos cifrados con SSL/TLS → seguros

**¿Por qué HTTPS es crítico?**
- Protege credenciales en tránsito
- Evita man-in-the-middle attacks
- Requerido para muchas APIs modernas
- Google penaliza sitios sin HTTPS en SEO

**Obtener certificado:**
- **Let's Encrypt**: Gratuito, automatizado
- **Cloudflare**: Proxy con SSL gratuito
- **Certificados comerciales**: Para empresas grandes

---

## 16. DESPLIEGUE Y PRODUCCIÓN

### 16.1. Diferencias Desarrollo vs Producción

**Entorno de Desarrollo:**
- **Debug activado**: Muestra errores detallados
- **Auto-reload**: Reinicia al cambiar código
- **BD local**: SQLite, datos de prueba
- **Sin HTTPS**: Localhost en HTTP
- **Logs verbosos**: Toda la información

**Entorno de Producción:**
- **Debug DESACTIVADO**: Errores genéricos (no revelar
estructura)
- **Sin auto-reload**: Estabilidad
- **BD robusta**: PostgreSQL, MySQL
- **HTTPS obligatorio**: Seguridad
```

- **Logs filtrados**: Solo errores críticos

16.2. Servidores WSGI de Producción

¿Por qué NO usar el servidor de desarrollo de Flask?
- Monohilo (una petición a la vez)
- Lento
- Inseguro
- No diseñado para carga real

Alternativas de producción:

Gunicorn (Green Unicorn):
- **Pre-fork worker model**: Múltiples procesos
- Fácil de configurar
- Estándar en Linux

uWSGI:
- Más complejo pero más potente
- Muchas opciones de configuración
- Mejor rendimiento en algunos casos

Waitress:
- Multiplataforma (Windows también)
- Más simple que uWSGI
- Bueno para proyectos pequeños

16.3. Arquitectura de Producción Típica

Stack completo:
```
Internet
    ↓
[Cloudflare/CDN] → Caché, SSL, protección DDoS
    ↓
[Nginx] → Reverse proxy, archivos estáticos, balanceo
    ↓
[Gunicorn] → WSGI server, múltiples workers
    ↓
```

```
[Flask App] → Tu código
    ↓
[PostgreSQL] → Base de datos
```

Función de cada capa:
- **CDN**: Distribución global de contenido estático
- **Nginx**: Primera línea, maneja SSL, compresión, caché
- **Gunicorn**: Ejecuta tu aplicación Python eficientemente
- **Flask**: Lógica de negocio
- **PostgreSQL**: Persistencia fiable y escalable

16.4. Variables de Entorno y Configuración

Problema:
Credenciales (BD, APIs) hardcodeadas en código → riesgo de seguridad si se sube a Git.

Solución:
- **Variables de entorno**: Sistema operativo proporciona valores
- **.env archivo** (desarrollo, nunca en Git)
- **Secrets managers** (producción): AWS Secrets Manager, HashiCorp Vault

Configuración por entorno:
```
DESARROLLO: DEBUG=True, DB=local.db
STAGING: DEBUG=False, DB=staging.postgres
PRODUCCIÓN: DEBUG=False, DB=prod.postgres (replicado)
```

16.5. Contenedores y Docker

Concepto:
Docker empaqueta la aplicación con TODAS sus dependencias en un "contenedor" portátil.

Ventajas:

- **Consistencia**: "Funciona en mi máquina" → funciona en todas
- **Aislamiento**: Cada app en su contenedor
- **Despliegue simple**: Mismo contenedor en desarrollo, staging, producción
- **Escalabilidad**: Fácil replicar contenedores

Flujo típico:
1. Crear **Dockerfile** (receta del contenedor)
2. Build → imagen Docker
3. Push → Registro (Docker Hub, AWS ECR)
4. Pull y ejecutar en servidor de producción

---

17. BASES DE DATOS Y ORM

17.1. SQL vs NoSQL

SQL (Relacional):
- **Estructura**: Tablas con relaciones definidas
- **Esquema**: Rígido, predefinido
- **Ejemplos**: PostgreSQL, MySQL, SQLite
- **Ideal para**: Datos estructurados, transacciones, integridad referencial
- **Consultas**: SQL estándar

NoSQL (No-relacional):
- **Estructura**: Documentos, key-value, grafos, columnas
- **Esquema**: Flexible, dinámico
- **Ejemplos**: MongoDB (documentos), Redis (key-value), Neo4j (grafos)
- **Ideal para**: Datos no estructurados, escalabilidad horizontal, velocidad
- **Consultas**: Específicas de cada BD

17.2. ORM (Object-Relational Mapping)

Concepto:

Capa que **traduce entre objetos Python y tablas SQL**, permitiendo trabajar con BD usando código Python puro, sin escribir SQL.

Ventajas:
- **Productividad**: Código más rápido y legible
- **Portabilidad**: Cambiar de BD sin cambiar código
- **Seguridad**: Previene inyección SQL automáticamente
- **Mantenibilidad**: Código más limpio

Desventajas:
- **Abstracción**: Puede ocultar problemas de rendimiento
- **Curva de aprendizaje**: Necesitas entender el ORM
- **Limitaciones**: Queries muy complejas pueden requerir SQL crudo

ORMs principales:
- **Django ORM**: Integrado en Django, potente, muchas funcionalidades
- **SQLAlchemy**: El más potente de Python, flexible, usado con Flask
- **Peewee**: Ligero, simple, ideal para proyectos pequeños

17.3. Migraciones de Base de Datos

Problema:
Cambios en modelos (agregar campo, nueva tabla) deben reflejarse en la BD.

Solución: Migraciones
Scripts que **transforman el esquema de BD** de un estado a otro, de forma controlada y reversible.

Herramientas:
- **Django Migrations**: Integrado, automático
- **Alembic**: Para SQLAlchemy (usado con Flask-Migrate)

Ventajas:
- **Control de versiones de BD**: Cada cambio registrado

- **Colaboración**: Todo el equipo sincronizado
- **Rollback**: Revertir cambios si algo falla
- **Automatización**: Aplicar cambios en todos los entornos

---

18. TESTING (Pruebas)

18.1. ¿Por qué Testing?

Beneficios:
- **Confianza**: Saber que el código funciona
- **Refactoring seguro**: Cambiar sin romper
- **Documentación viva**: Tests muestran cómo usar el código
- **Menos bugs**: Detectar problemas temprano
- **Velocidad**: Automatización más rápida que testing manual

18.2. Tipos de Tests

1. Unit Tests (Pruebas Unitarias):
- Testean **componentes individuales** (funciones, clases)
- Aislados, rápidos
- Mayor cantidad de tests

2. Integration Tests (Pruebas de Integración):
- Testean **interacción entre componentes** (ruta + BD)
- Más lentos, más complejos
- Cantidad media

3. End-to-End Tests (Pruebas de Extremo a Extremo):
- Testean **flujo completo** como usuario real
- Más lentos, más frágiles
- Menor cantidad, solo para flujos críticos

Pirámide de Testing:
```
    /\
   /E2E\      ← Pocos
  /______\
```

```
 /  INT  \    ← Medios
/_________\
/  UNIT   \   ← Muchos
/__________\
```

18.3. Testing en Flask

pytest: Framework de testing más popular en Python.

Flask Test Client: Simula peticiones HTTP sin levantar servidor:

- Permite testear rutas
- Verifica respuestas
- Comprueba redirecciones

Fixtures: Configuraciones reutilizables (BD de prueba, cliente de test, datos de ejemplo).

---

19. CONCEPTOS FINALES CLAVE

19.1. DRY (Don't Repeat Yourself)

Principio: Cada pieza de conocimiento debe tener una **representación única y autoritativa** en el sistema.

Aplicación:

- Herencia de plantillas (no duplicar header/footer)
- Funciones reutilizables (no copiar-pegar código)
- Configuración centralizada

19.2. Separation of Concerns (Separación de Responsabilidades)

Principio: Diferentes aspectos del sistema deben manejarse en módulos separados.

Aplicación:

- MVC/MVT: Separar datos, lógica, presentación
- Arquitectura 3 capas
- Blueprints/Apps para módulos funcionales

19.3. Idempotencia

Concepto: Una operación es **idempotente** si ejecutarla múltiples veces produce el mismo resultado que ejecutarla una vez.

Aplicación:

- GET, PUT, DELETE → Idempotentes
- POST → NO idempotente (crear múltiples recursos)

Importancia: Reintentar operaciones idempotentes es seguro → robustez ante fallos de red.

19.4. Escalabilidad

Escalabilidad Vertical:

- Más recursos a una máquina (más CPU, RAM)
- Límite físico
- Más caro

Escalabilidad Horizontal:

- Más máquinas/instancias
- Casi ilimitado
- Requiere arquitectura adecuada (stateless, balanceador)

Flask/Django: Diseñados para escalar horizontalmente con configuración correcta.